

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – INCIDENT RESPONSE
(MONITORING & MITIGATION)

Course Code:	CSA5803	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO5 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 3
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	Develop Incident Response Plan, Analyze Security Monitoring Dashboard, Recommend Mitigation Strategies and Technical Justification.		
Student Name:	Harish T	Reg. No.:	192521373
Section / Batch:	B Tech IT 2025	Date:	27/08/26

Instructions to Students

- Develop an incident response plan based on the given security incident scenario and identified findings.
- Analyze the provided security monitoring dashboard, alerts, logs and indicators to determine incident status and affected assets.
- Recommend prioritized mitigation strategies that directly address the identified threats, vulnerabilities and incident impact.
- Provide clear technical justification for each major response and mitigation decision using evidence from the scenario.
- Present the response plan, dashboard analysis and recommendations clearly and professionally within the prescribed submission format.

Question Type	Focus Area	Questions	Marks
INCIDENT RESPONSE	Monitoring & Mitigation	Q1	Q1 –100
GRANDTOTAL:		100 Marks	

Code of Conduct

I Harish T (192521373)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Harish T

ASSESSMENT TOOL 3 – INCIDENT RESPONSE (MONITORING & MITIGATION)

CSA5803 – DevSecOps Engineering and Application Security

Analyze an Airline Reservation System experiencing automated booking requests and service degradation. Develop an incident response plan, recommend mitigation strategies, and justify bot protection, rate limiting, autoscaling, and DDoS controls.

Q1. Incident Response, Monitoring and Mitigation Plan

1. Understanding of the Scenario

The airline reservation system is experiencing automated booking requests together with service degradation. This should be treated as an availability and application-security incident because abnormal request activity is affecting the ability of the reservation platform to serve legitimate customers. The reservation platform is a business-critical system because passengers depend on it to search flights, check availability, create reservations and complete transactions. Any degradation therefore has a direct technical and operational impact.

The incident should not be treated as a simple traffic spike without investigation. Automated requests can consume application workers, CPU, memory, network connections, API capacity and database resources. If these resources become saturated, legitimate customer requests may wait in queues, time out or fail. At the same time, the system may attempt to scale, increasing infrastructure cost.

The scenario establishes automated booking requests and service degradation, but it does not by itself establish the exact source, request volume, attack tool or whether a DDoS attack is occurring. Therefore, the response must be evidence-driven. The incident team should use logs, monitoring dashboards, WAF/CDN records and infrastructure metrics to determine the actual cause and severity.

2. Identification of Key Issues

- Automated request generation against reservation-related endpoints.
- Possible exhaustion of application CPU, memory, workers, connections or queues.
- Possible database overload if every request triggers expensive queries.
- Increased latency, timeouts and application errors.
- Reduced ability of legitimate passengers to complete bookings.
- Potential infrastructure-cost increase if autoscaling responds to abusive traffic.
- Possible weaknesses in bot protection, rate limiting, caching or edge protection.
- Risk of cascading failure if application pressure propagates to dependent services.

The most important issue is loss or degradation of availability. The response should therefore prioritize service continuity while preserving sufficient evidence to determine root cause. The assessment rubric emphasizes understanding the scenario, identifying key issues, applying concepts, making justified decisions and presenting the response clearly.

3. Affected Assets and Business Impact

- Customer-facing reservation website and booking interface.
- Flight-search and availability APIs.
- Booking creation and reservation APIs.
- CDN, WAF, API gateway and load balancer.
- Application servers, containers or cloud instances.

- Databases, caches and connection pools.
- Authentication/session services and payment-related components where applicable.
- Monitoring, logging and security systems.

These assets should be investigated together because a failure in one layer can create pressure in another. For example, repeated availability requests may increase application processing and database queries. If the database becomes slow, application threads can remain occupied longer, which further increases the request queue. The business consequences include failed or abandoned bookings, customer complaints, possible revenue loss and increased operational cost.

4. Incident Sequence

- Automated clients send repeated booking-related requests.
- Reservation services accept and process the requests, consuming resources.
- CPU, memory, application workers, connections or database workload increase.
- Latency, timeouts and error rates rise.
- Legitimate customers experience degraded service.
- Monitoring and security systems detect abnormal traffic and service-health changes.
- The response team investigates and applies targeted containment.
- Service health is verified during recovery and controls are improved afterward.

5. Security Monitoring Dashboard Analysis

The security monitoring dashboard is central to identifying the incident. The team should establish a normal baseline and compare current values against it. A high request rate alone does not prove malicious activity because airline traffic can legitimately increase during holidays, promotions or fare releases. Strong evidence is the combination of abnormal request behavior and measurable service degradation.

- Request rate – compare requests per second with normal traffic.
- Endpoint concentration – identify whether booking or other expensive endpoints receive disproportionate traffic.
- HTTP status codes – examine increases in 4xx, 5xx and timeout responses.
- Latency – compare current response times with normal service levels.
- CPU and memory – identify application resource saturation.
- Worker and connection utilization – check thread pools, API connections and database connections.
- Database metrics – examine query rate, slow queries, connections and cache behavior.
- WAF/CDN indicators – review blocked, challenged and suspicious requests.
- Source and session patterns – identify repetitive behavior and unusual frequency.
- Autoscaling activity – determine whether added capacity actually improves service health.

A useful monitoring timeline is: abnormal traffic starts → request rate rises → resources become saturated → latency/errors increase → mitigation is activated → abusive traffic falls → resource pressure decreases → service recovers. This correlation provides evidence that the traffic is responsible for the degradation.

6. Evidence Collection

Evidence should be collected before major changes where practical so that the incident can be reconstructed later. Web and API logs should contain timestamps, endpoints, methods, status codes and response times. WAF/CDN logs can show blocked or challenged traffic. Load-balancer information can show incoming connections and backend health. Application and database monitoring can identify which component became the limiting factor.

The team should preserve relevant logs, record mitigation timestamps and correlate events across systems. This is important because it allows the response team to demonstrate whether a mitigation actually reduced the problem. The evidence can also support the final root-cause analysis and help prevent recurrence.

7. Root Cause Analysis

The most reasonable immediate-cause hypothesis from the scenario is that automated booking requests are generating excessive application workload, creating resource pressure and resulting in service degradation. The causal chain is: automated requests → increased API/application workload → resource pressure and queueing → increased latency/errors → disruption for legitimate customers.

The final root cause should be confirmed from evidence. The investigation should determine why the automated requests were able to generate such a large workload. Possible contributing conditions include insufficient bot protection, weak endpoint-specific rate limits, expensive reservation queries, inadequate caching, weak concurrency controls, insufficient capacity planning or inadequate DDoS protection. These should be recorded as contributing factors only when supported by evidence.

A strong incident report should distinguish confirmed observations, technical root cause, contributing weaknesses, business impact and mitigation effectiveness. This provides a clear connection between the incident, its causes and the recommended controls.

8. Risk Assessment

- Critical – service availability: continued overload may make the reservation service unavailable.
- Critical – legitimate customer disruption: passengers may be unable to search, reserve or complete transactions.
- High – application/API exhaustion: workers, CPU, memory and connections may be consumed.
- High – database exhaustion: repeated expensive operations may overload shared backend resources.
- High – infrastructure cost: uncontrolled autoscaling may create additional capacity without solving abuse.
- High – cascading failure: overload may propagate to dependent services.
- Medium-High – false positives: aggressive bot blocking or rate limits may affect legitimate customers.

Risk should be prioritized using likelihood, severity and technical or operational impact. The highest priorities are availability and legitimate customer disruption because these are direct consequences of the stated service degradation.

9. Incident Response Plan – Preparation

Preparation establishes the capabilities needed before an incident occurs. The airline should maintain centralized application, infrastructure and security logs; real-time dashboards; WAF/CDN policies; bot-management capability; API rate-limit configurations; autoscaling policies and DDoS-provider integration. Responsibilities should be assigned to security, application, infrastructure, database and business teams.

Normal traffic and resource baselines should be documented. Because airline demand naturally changes, alerts should use multiple indicators rather than one threshold. A well-prepared organization can detect unusual request patterns earlier and can activate controls without making untested emergency changes.

10. Identification and Triage

When an alert occurs, the incident commander should confirm the time of degradation, affected services and severity. The team should compare request volume with historical baselines and determine which endpoints are receiving abnormal traffic. CPU, memory, application queues, database connections and error rates should be reviewed at the same time.

The team should classify traffic as legitimate, automated, malicious or mixed. This is essential because blocking all high-volume traffic could harm genuine passengers. Evidence such as repeated request patterns, unusual request frequency, suspicious session behavior and bot-management signals should be used for classification.

11. Containment

- Apply targeted rate limits to high-risk and resource-intensive reservation endpoints.
- Enable or tighten bot-management and WAF rules.
- Challenge suspicious clients where appropriate instead of broadly blocking shared networks.
- Protect expensive booking operations with authentication, quotas and concurrency limits.
- Reduce unnecessary database work through caching and query optimization.
- Prioritize essential booking functions.
- Activate stronger DDoS protection when evidence indicates large-scale flooding.

Containment should be gradual and measurable. After each major control is applied, monitor latency, error rates, booking success and origin traffic. If a mitigation reduces suspicious traffic but also prevents many legitimate customers from booking, it must be adjusted.

12. Eradication and Recovery

After immediate containment, the team should correct the weaknesses that allowed excessive workload. This may include improving bot detection, introducing endpoint-specific rate limits, optimizing APIs and queries, adding caching, strengthening authentication and protecting database connections.

Recovery should be gradual. Emergency controls should remain active while request volume, latency, error rates and resource utilization are monitored. Normal thresholds can be restored only after stable operation is confirmed. Recovery means more than bringing servers online; legitimate customers must also be able to complete reservations successfully.

13. Post-Incident Review

The review should record detection time, investigation time, containment time, recovery time, customer impact, false positives, infrastructure cost and mitigation effectiveness. The team should identify what was detected early, what was missed and which controls worked.

The lessons should be converted into updated runbooks, better alert thresholds, improved bot rules, refined rate limits, safer autoscaling policies and stronger DDoS procedures. This turns the incident into an opportunity for continuous security and reliability improvement.

14. Bot Protection – Detailed Justification

Bot protection is directly justified because automated booking requests are the central behavior described in the scenario. Bot-management controls can analyze behavioral and request-level signals to identify traffic that does not resemble normal passenger interaction. Depending on confidence, suspicious clients can be allowed, challenged, throttled or blocked.

The major advantage is precision. Simple IP blocking can affect legitimate customers because many users may share an address through mobile carriers, corporate networks or NAT. Behavioral controls can consider multiple signals and therefore provide more targeted enforcement.

Bot protection should be strongest around high-value operations such as repeated availability searches and booking attempts. The airline should monitor false positives, challenge success and legitimate booking success. The goal is to stop abusive automation while preserving access for genuine passengers.

15. Rate Limiting – Detailed Justification

Rate limiting places an upper bound on request frequency for an appropriate identity such as an IP address, account, session, API key or token. It prevents a client from generating unlimited API workload. Limits should be endpoint-specific because different reservation functions have different legitimate usage patterns.

Search operations may legitimately require repeated requests, while booking creation or payment operations should generally have stricter limits. A client exceeding a threshold can receive HTTP 429, and the event can be logged for investigation. Enforcement at the CDN, WAF or API gateway is preferable because excessive requests can be rejected before they consume expensive application and database resources.

Rate limiting and bot protection complement each other. Bot protection identifies suspicious automation, while rate limiting limits how much damage that traffic can cause. Together they provide stronger protection than either control alone.

16. Autoscaling – Detailed Justification

Autoscaling dynamically increases application capacity when demand rises. This is useful for an airline because legitimate traffic can change quickly. Additional instances can distribute workload, reduce queues and improve availability during genuine demand peaks.

Autoscaling is not a replacement for security controls. If automated traffic continually increases, autoscaling may repeatedly create additional instances, increasing cost while a database or another shared dependency remains overloaded. Autoscaling should therefore have sensible minimum and maximum capacity and should work together with bot protection, rate limiting, caching and downstream safeguards.

Autoscaling should be viewed as a resilience mechanism. It is most effective after unwanted traffic has been controlled and the remaining workload is legitimate or manageable.

17. DDoS Controls – Detailed Justification

DDoS controls protect against distributed traffic capable of exhausting network, transport or application resources. Upstream or network-level protection can help absorb large traffic volumes, while CDN and WAF filtering can prevent unwanted HTTP traffic from reaching the origin.

Application-layer abuse can use valid-looking requests that are individually expensive. Therefore, DDoS controls cannot replace bot management or rate limiting. They must be combined with them. The airline should define escalation thresholds, activation procedures, responsible teams and recovery steps, and should test those procedures before an actual outage.

18. Technical Justification of the Four Controls

Control	Primary purpose	Justification
Bot protection	Detect and control automation	Directly addresses automated booking behavior and reduces suspicious traffic before expensive processing.
Rate limiting	Restrict request frequency	Prevents clients or identities from consuming unlimited API/application capacity.
Autoscaling	Add capacity	Maintains availability during legitimate demand peaks and absorbs controlled residual workload.
DDoS controls	Protect edge/network/origin	Filters or absorbs large-scale traffic before it overwhelms infrastructure.

19. Recommended Layered Security Architecture

The recommended request path is: Internet → DDoS protection → CDN/WAF → bot-management controls → API gateway/rate limiter → load balancer → autoscaled reservation services → cache/database protection.

This follows defense in depth. DDoS protection handles large-scale traffic at the edge. CDN/WAF filtering

removes unsuitable traffic. Bot protection identifies automation. The API gateway enforces endpoint-specific limits. The load balancer distributes allowed traffic. Autoscaling provides capacity for legitimate demand. Caching and database controls reduce backend pressure.

Monitoring should exist at every layer. The team should see incoming traffic, blocked traffic, challenged traffic, rate-limited traffic, origin requests, application latency, errors, scaling activity and database utilization. This provides evidence that each mitigation is performing its intended function.

20. Prioritized Mitigation Strategy

- Priority 1 – Restore availability and identify the overloaded component.
- Priority 2 – Reduce automated workload using bot protection and targeted rate limiting.
- Priority 3 – Protect the network and application edge using DDoS controls.
- Priority 4 – Maintain resilience using autoscaling with safe maximum limits.
- Priority 5 – Reduce backend work through caching, query optimization and concurrency controls.
- Priority 6 – Prevent recurrence through improved monitoring, alerting, runbooks and testing.

This order is important because adding capacity alone does not eliminate abusive traffic. Unwanted workload should first be controlled, while autoscaling provides resilience for the legitimate workload that remains.

21. Final Recommendation

The airline should implement a layered, evidence-driven incident-response strategy. The first step is to confirm the traffic pattern and identify the resource responsible for degradation. Immediate containment should use targeted bot protection and endpoint-specific rate limiting. DDoS mitigation should protect the edge when traffic scale or distribution requires it. Autoscaling should maintain capacity for legitimate demand, but it must have safe limits and downstream safeguards.

Success should be measured using restored response time, reduced errors, stable application and database utilization, improved legitimate booking success, reduced abusive traffic and controlled infrastructure cost. The post-incident review should use these measurements to improve security monitoring, application design and future response procedures.

22. Conclusion

The airline reservation incident requires coordinated monitoring, incident response, root-cause analysis and mitigation. Automated requests can consume application and backend resources and degrade service, but the exact root cause should be confirmed through evidence. A structured response therefore begins with preparation and identification, continues through triage and containment, and ends with recovery and lessons learned.

Bot protection directly addresses automation, rate limiting restricts request frequency, autoscaling provides resilience for legitimate demand, and DDoS controls protect the edge and origin from large-scale traffic. No single control is sufficient. Together, these controls provide defense in depth, reduce recurrence risk and help maintain reliable reservation services for legitimate airline customers.